Transformers & Hugging Face

Jour 4 — Après-midi

Julien Rolland

Formation M2 Développement Fullstack

Jour 4

Plan du module

1. Du contexte à l'attention
2. L'architecture Transformer
3. Hugging Face en pratique
4. Vers les LLM modernes

Le problème du contexte

La polysémie en pratique

- « La **souris** mange le fromage. »
- « Ma **souris** d'ordinateur est cassée. »

Dans Word2Vec, « souris » n'a **qu'un seul vecteur** — une moyenne des deux sens.

Les autres limites (rappel J4-AM)

- Embeddings **figés** après entraînement
- Ignorent l'**ordre** des mots dans la phrase
- Biais hérités du corpus d'entraînement

Le besoin : embeddings contextuels

Le vecteur d'un mot doit être calculé **à la volée** en fonction de ses voisins.

« La souris **mange**... » → $\vec{v}_{\text{animal}}$

« Ma souris d'**ordinateur**... » → $\vec{v}_{\text{outil}}$

Solution

Le **mécanisme d'attention** — cœur du Transformer.

L'intuition de l'attention

Principe & exemple

Chaque mot « vote » pour ceux qui l'aident à se définir.

« L'animal ... car **il** était fatigué. »

Distribution d'attention de « il » :

- **animal** : **90 %**
- fatigué : 6 %
- autres mots : 4 %

Avantage : parallélisme

Contrairement aux RNN, l'attention traite **tous les mots simultanément** — GPU.

Distribution d'attention de « **il** »

90 % **animal** — co-référence directe

6 % **fatigué** — trait partagé

4 % autres mots (négligeable)

Interprétation

Le modèle « sait » que « il » réfère à « animal » **grâce au contexte** global — sans règle grammaticale explicite.

La formule de l'attention

Mécanisme Query / Key / Value

- Q (*Query*) : « Ce que je **cherche**. »
- K (*Key*) : « Ce que j'**offre**. »
- V (*Value*) : « L'information que je **contiens**. »

Équation

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

- $QK^\top$: score de pertinence (mot, mot)
- $\sqrt{d_k}$: normalisation (gradients stables)
- Softmax $\rightarrow$ poids $\in [0, 1]$, somme $= 1$

Multi-Head Attention

On lance h attentions en **parallèle** sur des sous-espaces distincts :

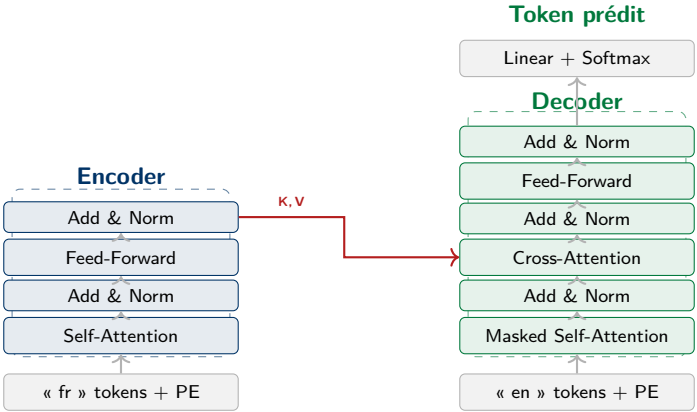
- Tête 1 : relations **grammaticales**
- Tête 2 : relations **sémantiques**
- Tête 3 : relations de **coréférence**
- ...

Les sorties sont **concaténées** puis projetées.

Note « Geek »

C'est une **recherche floue** dans une base de données vectorielle : Q est la requête, K l'index, V le résultat.

L'architecture Transformer (Vaswani et al., 2017)



Encoder, Decoder, Seq2Seq

Encoder (ex: BERT)

+ Lit la phrase **dans les deux sens**
+ Embeddings contextuels riches
- Idéal : **classification**, NER, question-answering

Decoder (ex: GPT)

+ Génère mot après mot (*autoregressive*)
- Attention **masquée** : ne voit que le passé
- Idéal : **génération** de texte, chatbots

Encoder-Decoder (ex: T5)

+ Encode la source, décode la cible
- Idéal : **traduction**, résumé, reformulation

BERT, RoBERTa

Encoder
Self-Attention (bidirectionnel)

Classification, NER

GPT-2/3/4, LLaMA

Decoder
Masked Self-Attention

Génération

T5, BART, mBART

Encoder + Decoder
Cross-Attention

Traduction, Résumé

Le paradigme du Transfer Learning

Pre-training — la fondation

- Entraînement massif sur **tout Internet** (Wikipédia, livres, code...)
- Objectif : **prédire le token masqué** (BERT) ou le token suivant (GPT)
- Coût : millions d'euros de calcul GPU
- Résultat : le modèle apprend la **« langue »** — grammaire, faits, raisonnement

Fine-tuning — l'adaptation

On ajuste le modèle sur une **tâche métier** :

- Classification de tickets support
- Résumé automatique de rapports
- Détection de sentiment client

Quelques dizaines de lignes de code, quelques **minutes** de calcul → niveau état de l'art.

Pourquoi ça marche ?

Les couches inférieures encodent des connaissances **générales** réutilisables ; seule la tête de classification change.

Anatomie du Fine-Tuning

Les 3 composants obligatoires

1. **Tokenizer** : identique au modèle pré-entraîné
2. **Model Head** : couche de sortie remplacée (ex: 30 000 → 2 classes)
3. **Trainer** : boucle d'entraînement optimisée

```
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
)
model = AutoModelForSequenceClassification\
  .from_pretrained("camembert-base",
                   num_labels=2)
tokenizer = AutoTokenizer\
  .from_pretrained("camembert-base")
```

Pourquoi le Tokenizer doit coïncider ?

Le modèle a appris des embeddings pour des IDs précis. Si l'on utilise un tokenizer différent, les IDs ne correspondent plus aux poids — le modèle produit du bruit.

Alternatives légères

- **LoRA / QLoRA** : $< 1\,\%$ des params
- **Prompt-tuning** : tokens « soft »
- **Zero-shot** : sans fine-tuning

Hugging Face — Le Hub & `datasets`

The Hub — le GitHub des modèles

- $>$ 900 000 modèles prêts à l'emploi
- Hébergement gratuit des poids
- Versionning, fiches modèles (*Model Cards*), benchmarks intégrés
- Chargement direct depuis le code :

```python
from transformers import pipeline
clf = pipeline("sentiment-analysis",
    model="distilbert-base-uncased-"
          "finetuned-sst-2-english")
```

`datasets` — toutes les données

Accès instantané à des milliers de bases :

- IMDb, Wikipedia, GLUE, SQuAD...
- Streaming pour les grands corpus
- Compatibles PyTorch et TensorFlow

```python
from datasets import load_dataset

ds = load_dataset("glue", "sst2")
# DatasetDict({train: 67349, ...})

print(ds["train"][0])
# {'sentence': 'hide new secrets',
#  'label': 0}
```

Hugging Face — `transformers` & écosystème

`transformers` — interface unifiée

Même API quel que soit l'architecture :

- **Encodeurs** : BERT, RoBERTa, CamemBERT
- **Décodeurs** : GPT-2, LLaMA, Mistral
- **Seq2Seq** : T5, BART, Whisper, CLIP...

`Auto*` : détecte l'architecture automatiquement.

Chargement universel

`AutoTokenizer`, `AutoModel`, `AutoModelForSequenceClassification`, `AutoModelForCausalLM`...

Écosystème complet

- `tokenizers` : tokeniseurs ultra-rapides (Rust)
- `evaluate` : métriques standardisées (accuracy, F1, BLEU...)
- `accelerate` : multi-GPU/TPU sans changer le code
- `PEFT` : fine-tuning efficace (LoRA, QLoRA)
- `optimum` : export ONNX, quantization

Vision d'ensemble

Hub → `datasets` → `transformers` → `Trainer` → Hub

Un cycle complet : charger, entraîner, publier.

La `pipeline` API — le mode « Product »

Concept

Abstraction totale : téléchargement du modèle, tokenisation, inférence, décodage — tout en **une seule ligne**.

```python
from transformers import pipeline

clf = pipeline("sentiment-analysis")
print(clf("Ce cours est dense !"))
# [{'label':'POSITIVE', 'score':0.98}]

trad = pipeline("translation",
    model="Helsinki-NLP/opus-mt-fr-en")
print(trad("Bonjour le monde"))
# [{'translation_text':'Hello World'}]
```

Tâches disponibles

- `"sentiment-analysis"`
- `"text-generation"`
- `"summarization"`
- `"translation"`
- `"question-answering"`
- `"ner"` (entités nommées)

Intérêt terrain

C'est ce qu'un dev Fullstack utilisera **90 % du temps** en entreprise. Pas besoin de connaître l'architecture interne.

Le Trainer — boucle complète

```python
from transformers import (
    TrainingArguments, Trainer,
    DataCollatorWithPadding)

# dataset tokenise via .map() (cf. slide precedente)

args = TrainingArguments(
    output_dir="./results",
    num_train_epochs=3,
    per_device_train_batch_size=16,
    eval_strategy="epoch",
    load_best_model_at_end=True)

trainer = Trainer(
    model=model,
    args=args,
    train_dataset=ds["train"],
    eval_dataset=ds["test"],
    data_collator=DataCollatorWithPadding(
        tokenizer))

trainer.train()
```

Ce que gère le Trainer

- Shuffling, batching, padding dynamique
- Gradient accumulation
- Checkpoints + évaluation automatiques

Pour aller plus loin

- `compute_metrics` : F1 à chaque epoch
- `callbacks` : early stopping
- `accelerate` : multi-GPU

Séquence typique

`map()` → `Trainer()` → `train()` → `push_to_hub()`

Instruction Tuning & Alignement (RLHF)

Le problème du modèle de base

Un LLM pré-entraîné complète du texte brut ; il ne **suit pas les instructions** :

- Entrée : « Explique la photosynthèse. »
- Sortie : « Explique la photosynthèse. Exercice 3... » (continuation !)

Résultat

+ Suit les instructions précisément
+ Refuse les requêtes dangereuses
+ Répond de façon utile et concise

C'est ce qui transforme GPT-2 en ChatGPT, LLaMA en LLaMA-Chat / Mistral-Instruct.

Pipeline d'alignement

1. **SFT** : fine-tuning supervisé sur des paires question / réponse rédigées par des humains
2. **Reward Model** : apprend à préférer une bonne réponse sur une mauvaise
3. **RLHF** : le LLM est optimisé pour maximiser la récompense humaine

Variantes modernes

- **DPO** : plus stable que RLHF classique
- **Constitutional AI** (Anthropic) : l'IA se critique elle-même
- **RLAIF** : feedback automatique, sans annotation humaine

Prompt Engineering

Zero-shot vs Few-shot

Zero-shot : instruction seule, sans exemple.

Few-shot : 2–5 exemples dans le prompt.

```
# Zero-shot
prompt = """Classe ce tweet :
POSITIF ou NEGATIF.
Tweet: "Ce cours est top !"
Sentiment:"""

# Few-shot
prompt = """Tweet: "J'adore !" -> POSITIF
Tweet: "Nul !"   -> NEGATIF
Tweet: "Sympa mais long." ->"""
```

Chain-of-Thought (CoT)

Ajouter « Raisonne étape par étape » améliore fortement les tâches logiques.

Quand fine-tuner plutôt que prompter ?

- Vocabulaire très spécialisé (médical, légal, code propriétaire)
- Fenêtre de contexte insuffisante
- Coût d'inférence à réduire
- Données confidentielles (pas d'API externe)

Règle pratique

Commencer par le prompting.

Fine-tuner si les limites sont atteintes — coût en temps, données et argent.

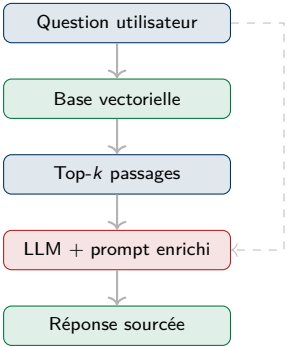
RAG — Retrieval-Augmented Generation

Limites des LLM seuls

- Connaissance figée à la date d'entraînement
- Hallucinent des faits non vus
- Ne connaissent pas vos données internes (docs techniques, base clients. . .)

Solution : RAG

1. **Indexer** les documents en embeddings dans une base vectorielle
2. À chaque question : **chercher** les k passages les plus proches
3. **Injecter** ces passages dans le prompt
4. Le LLM **répond** en se basant sur des sources vérifiées



Outils courants

LangChain, LlamaIndex, Chroma, FAISS, pgvector (PostgreSQL)

Récapitulatif — Ce qu'on a vu

Attention & Transformer

- Polysémie → besoin d'embeddings contextuels
- Mécanisme QKV : score de pertinence + softmax
- Multi-Head : plusieurs types de relations en parallèle
- Architecture : Encoder (BERT), Decoder (GPT), Seq2Seq (T5)

Hugging Face en pratique

- Hub : 900k+ modèles, versionning, Model Cards
- `datasets` + `transformers` : une ligne pour charger données et modèles
- `pipeline` : inférence clé en main (90 % des cas)
- `Trainer` : boucle d'entraînement complète
- Écosystème : tokenizers, evaluate, accelerate, PEFT

Transfer Learning

- Pre-training : apprendre la langue sur tout Internet
- Fine-tuning : adapter en quelques minutes sur une tâche
- Alternatives légères : LoRA, prompt-tuning, zero-shot

LLM modernes

- RLHF : aligner le modèle sur les préférences humaines
- Prompt engineering : zero-shot, few-shot, CoT
- RAG : ancrer les réponses dans des documents réels